САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бэк-энд разработка

Отчет

Практическая работа "ДЗ 5: Реализация межсервисного взаимодействия посредством очередей сообщений"

Выполнил:

Недиков Михаил

БР1.2

Проверил:
Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Задание:

- подключить и настроить rabbitMQ/kafka;
- реализовать межсервисное взаимодействие посредством rabbitMQ/kafka.

Ход работы

В микросервисный проект, спроектированный в ДЗ4 и реализованный в ЛР2, добавлен асинхронный обмен сообщениями между сервисами. Для передачи сообщений использован брокер RabbitMQ.

RabbitMQ выбран благодаря простой настройке и достаточным для работы возможностям. Брокер запущен через Docker Compose: порт 5672 используется для AMQP, а 15672 — для Management UI.

События, реализованные в системе:

- Событие "booking_request.created" - издатель: Booking Service. Отправляется после успешного создания заявки на бронирование. Содержит данные: bookingRequestId, propertyId, tenantId, ownerId, comments.
- Событие "user.deleted" - издатель: Auth & User Service. Отправляется после успешного удаления пользователя через эндпоинт DELETE /users/me. Содержит userId.

Подписчики и их действия:

- Engagement Service подписан на "booking_request.created". При получении события автоматически создает conversation (чат) между

tenantId и ownerId, привязанный к propertyId. Используется существующий эндпоинт создания чата.

- Engagement Service подписан на "user.deleted". При получении события удаляет все отзывы пользователя (reviews), а также все чаты (conversations) и связанные сообщения (messages), где пользователь является участником.
- Booking Service подписан на "user.deleted". При получении события отменяет все активные бронирования пользователя (меняет статус на cancelled) и удаляет все заявки на бронирование пользователя.
- Property Service подписан на "user.deleted". При получении события находит все объявления пользователя и удаляет их вместе со связанными локациями, атрибутами и изображениями.

Что было добавлено в каждый сервис:

Общая инфраструктура: папка rabbitmq/ с файлами connection.ts (подключение к брокеру), publisher.ts (отправка событий), consumer.ts (прием и обработка событий), setup-consumers.ts (регистрация обработчиков).

В Auth Service: метод DELETE /users/me с отправкой события user.deleted.

В Booking Service: отправка события booking_request.created после создания заявки; consumer для user.deleted с логикой отмены бронирований.

В Property Service: consumer для user.deleted с логикой удаления объявлений.

В Engagement Service: consumer для booking_request.created с логикой создания чата; consumer для user.deleted с логикой удаления отзывов, чатов и сообщений.

Docker Compose: добавлен контейнер rabbitmq.

Переменные окружения: во все сервисы добавлена RABBITMQ_URL.

Файлы запуска (app.ts): во всех сервисах добавлены асинхронное подключение к RabbitMQ, регистрация обработчиков и запуск consumer'ов при старте приложения.

Результаты тестирования:

Создание заявки на бронирование через POST /booking-requests. В логах Engagement Service появилось сообщение о получении события и создании чата. Запрос GET /conversations подтвердил наличие нового чата между арендатором и владельцем.

```
query: COMMIT
Event published: booking_request.created {
  bookingRequestId: 2,
  propertyId: 1,
  tenantId: 2,
  ownerId: 1,
  comments: 'tessst'
}
```

```
Created conversation 2 for booking request 2
query: SELECT "c"."id" AS "c_id", "c"."user1_id" AS "c_user1_id", "c"."user2_id" AS "c_user2_
ation" "c" WHERE "c"."user1_id" = $1 OR "c"."user2_id" = $1 -- PARAMETERS: [2]
```

Удаление пользователя через DELETE /users/me. В логах Auth Service зафиксирована отправка события. В логах Property Service, Booking Service и Engagement Service зафиксирована обработка события. Проверка через API подтвердила, что объявления, бронирования, заявки, отзывы, чаты и сообщения пользователя удалены из соответствующих сервисов.

Вывод

В результате выполнения домашней работы я познакомился с устройством RabbitMQ и смог внедрить его в свой проект. Был реализован весь необходимый функционал, добавлены обработчики для ряда асинхронных действий.